
웹프로그래밍

조별과제

202104183 김현중
202204191 김유빈
202304073 박찬영

소개

-수강신청 사이트 제작(강남대학교 수강신청 사이트 참조)



2025년 동계개설 수강신청

이름	■■■■■
학번	■■■■■
학부(과)/전공	소프트웨어전공
학년	2
주야	주간
최대신청학점	9

로그아웃

공지사항

교과목조회

수강신청

수강신청조회

이수구분 : 기초교양

원격강의 : ☒ 전체 ☐ Y ☐ N

교과목명 :

조회

개설과목 [2건]

이수구분	개설전공	학점	강의시수	학수번호	교과목명	영역	주행심역 량	신청인원	원격강의	강의시간 (시험시간)	강의계획서
기초교양	교양	2	2	ND01602	Academic English L&S(Intermediate)			9	N	09:20-11:00	보기
기초교양	교양	3	3	ND01607	컴퓨터프로그래 밍			9	N	15:00-17:30	보기

팀원 역할

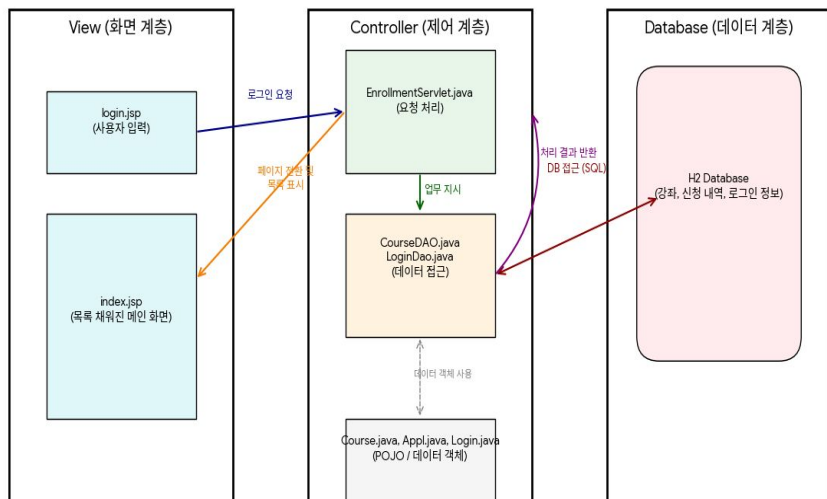
김현중 : 로그인, 회원가입 페이지 작성

김유빈 : 수강 신청 메인페이지 작성

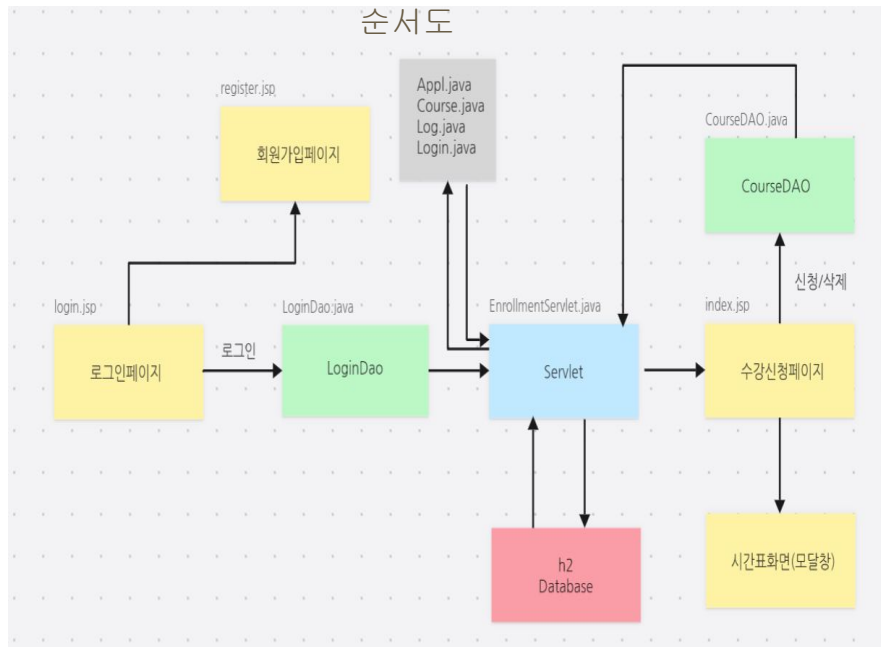
박찬영 : 시간표 팝업페이지 및 과목 DB 테이블 작성

사이트의 구성

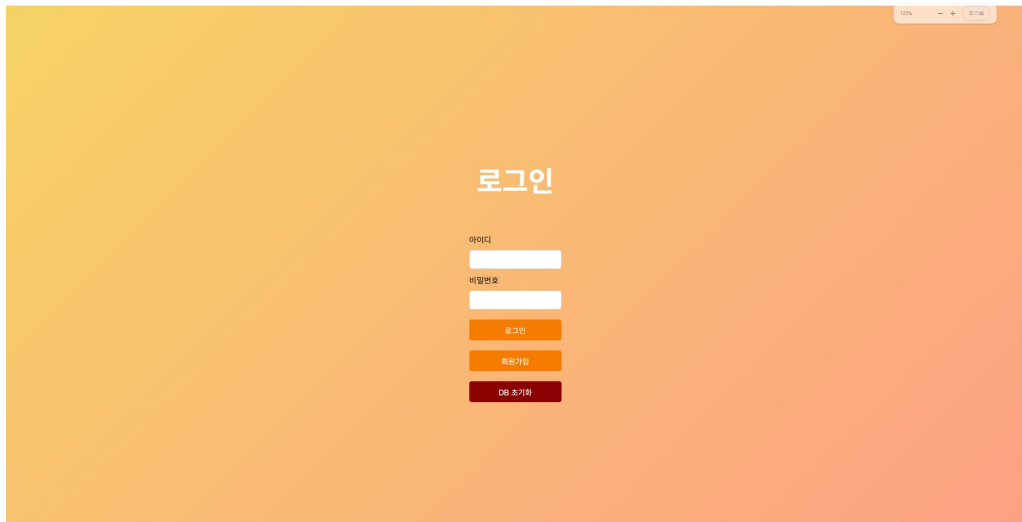
수강신청 시스템 아키텍처 (파일 연관성 중심)



수강신청
순서도



제작과정 - 로그인 페이지 (회원가입 버튼)



```
<form action="register.jsp" method="post">  
  <button type="submit">회원가입</button>  
</form>
```

제작과정 - 회원가입 페이지

회원가입

모든 정보를 입력해야 합니다.

이름

학번

학과

학년

아이디

비밀번호

가입하기

```
public void insert(Login s) {
    open();
    String sql = "INSERT INTO logins(STUDENT_ID, USERID, PASSWORD, NAME, MAJOR, GRADE) VALUES(?,?,?,?,?,?)";
    try {
        pstmt = conn.prepareStatement(sql);
        pstmt.setInt(1, s.getSTUDENT_ID());
        pstmt.setString(2, s.getUserID());
        pstmt.setString(3, s.getPASSWORD());
        pstmt.setString(4, s.getName());
        pstmt.setString(5, s.getMAJOR());
        pstmt.setInt(6, s.getGRADE());

        int count = pstmt.executeUpdate();

    } catch (Exception e) {
        System.out.println("ERROR(insert): insert failed for userId=" + s.getUserID());
        e.printStackTrace();
    } finally {
        close();
    }
}
```

```
<%
request.setCharacterEncoding("UTF-8");
String message = "";

if ("POST".equalsIgnoreCase(request.getMethod())) {
    String name = request.getParameter("name");
    String studentIdStr = request.getParameter("studentId");
    String major = request.getParameter("major");
    String gradeStr = request.getParameter("grade");
    String userId = request.getParameter("userId");
    String userPw = request.getParameter("userPw");

    if (name != null && !name.isEmpty() &&
        studentIdStr != null && !studentIdStr.isEmpty() &&
        major != null && !major.isEmpty() &&
        gradeStr != null && !gradeStr.isEmpty() &&
        userId != null && !userId.isEmpty() &&
        userPw != null && !userPw.isEmpty()) {

        try {
            int studentId = Integer.parseInt(studentIdStr);
            int grade = Integer.parseInt(gradeStr);

            Login newUser = new Login();
            newUser.setName(name);
            newUser.setSTUDENT_ID(studentId);
            newUser.setMAJOR(major);
            newUser.setGRADE(grade);
            newUser.setUserID(userId);
            newUser.setPassword(userPw);

            LoginDao dao = new LoginDao();
            dao.insert(newUser);

            response.sendRedirect("login.jsp");
            return;

        } catch (NumberFormatException e) {
            message = "학번과 학년은 숫자로 입력해야 합니다.";
        } catch (Exception e) {
            e.printStackTrace();
            message = "회원가입 중 오류가 발생했습니다.";
        }

    } else {
        message = "모든 정보를 입력해야 합니다.";
    }
}
%>
```

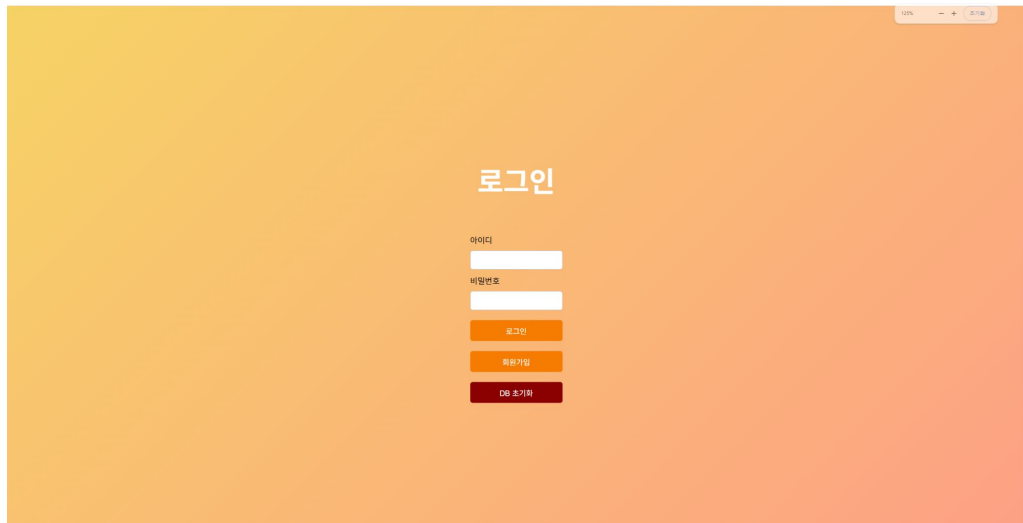
```
<h2>회원가입</h2>

<form action="register.jsp" method="post">
    <% if (!message.isEmpty()) { %>
        <p class="error"><%= message %></p>
    <% } %>

    이름
    <input type="text" name="name" required>
    학번
    <input type="text" name="studentId" required>
    학과
    <input type="text" name="major" required>
    학년
    <select name="grade" required>
        <option value="">선택</option>
        <option value="1">1학년</option>
        <option value="2">2학년</option>
        <option value="3">3학년</option>
        <option value="4">4학년</option>
    </select>
    아이디
    <input type="text" name="userId" required>
    비밀번호
    <input type="password" name="userPw" required>

    <button type="submit">가입하기</button>
</form>
```

제작과정 - 로그인 페이지 (DB초기화 버튼)



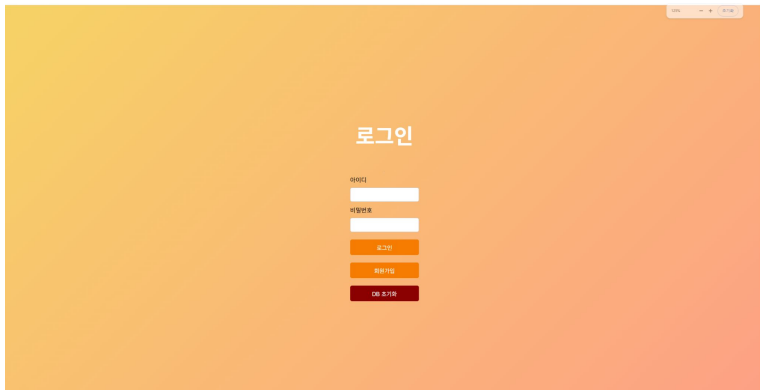
```
<form action="/project/enrollment?action=dbDelete" method="post">
  <button type="submit" class="reset-btn">DB 초기화</button>
</form>
```

```
case "dbDelete":
    daolog.deleteAll();
    response.sendRedirect("login.jsp");
    break;
```

```
public void deleteAll() {
    open();
    String sql = "DELETE FROM logins";

    try {
        pstmt = conn.prepareStatement(sql);
        pstmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
}
```

제작과정 - 로그인 페이지 (로그인 버튼)



```
public boolean loginCheck(String userId, String password) {
    open();
    boolean result = false;
    try {
        String sql = "SELECT * FROM logins WHERE USERID = ? AND PASSWORD = ?";
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, userId);
        pstmt.setString(2, password);

        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            result = true;
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return result;
}
```

```
public Login getUserById(String userId) {
    open();
    Login user = null;
    try {
        String sql = "SELECT * FROM logins WHERE USERID = ?";
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, userId);
        ResultSet rs = pstmt.executeQuery();

        if (rs.next()) {
            user = new Login();
            user.setSTUDENT_ID(rs.getInt("STUDENT_ID"));
            user.setUserID(rs.getString("USERID"));
            user.setPassword(rs.getString("PASSWORD"));
            user.setName(rs.getString("NAME"));
            user.setMAJOR(rs.getString("MAJOR"));
            user.setGRADE(rs.getInt("GRADE"));
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return user;
}
```

```
private LoginDao daoLog;

<c:if test="${not empty loginError}">
    <p class="error">${loginError}</p>
</c:if>

<form action="/project/enrollment?action=login" method="post">
    <label>아이디</label><br>
    <input type="text" name="userId" required><br>
    <label>비밀번호</label><br>
    <input type="password" name="userPw" required><br>
    <button type="submit">로그인</button>
</form>

String userId = request.getParameter("userId");
String userPw = request.getParameter("userPw");

if(daoLog.loginCheck(userId, userPw)) {
    Login user = daoLog.getUserById(userId);

    HttpSession session = request.getSession();
    session.setAttribute("userId", userId);
    session.setAttribute("loglog", user);

    allList(request, response);
    yesList(request, response);
    request.setAttribute("totalCount", dao.getTotalCredits());

    request.getRequestDispatcher("index.jsp").forward(request, response);
    return;
} else {
    request.setAttribute("loginError", "아이디 또는 비밀번호가 틀렸습니다.");
    request.getRequestDispatcher("login.jsp").forward(request, response);
    return;
}
```

제작과정 - 로그인 페이지 (로그인 버튼)

```
case "login":
    String userId = request.getParameter("userId");
    String userPw = request.getParameter("userPw");

    if(dao.loginCheck(userId, userPw)) {
        Login user = dao.getUserById(userId);

        HttpSession session = request.getSession();
        session.setAttribute("userId", userId);
        session.setAttribute("loglog", user);

        alllist(request, response);
        yeslist(request, response);
        request.setAttribute("totalCount", dao.getTotalCredits());

        request.getRequestDispatcher("index.jsp").forward(request, response);
        return;
    } else {
        request.setAttribute("loginError", "아이디 또는 비밀번호가 틀렸습니다.");
        request.getRequestDispatcher("login.jsp").forward(request, response);
        return;
    }
}
```

```
// 전체 강좌 신청 목록 select
public void alllist(HttpServletRequest request, HttpServletResponse response) {
    request.setAttribute("availableCourses", dao.notApplication());
}

// 강좌신청내역 목록 select
public void yeslist(HttpServletRequest request, HttpServletResponse response) {
    request.setAttribute("appliedCourses", dao.yesApplication());
}
```

KANGNAM UNIVERSITY

개설강좌신청

학수번호: 예 C001 학점: 전체 0000

2025년 월호로그레팅 수강신청

이름	학번	학점	신청사건	학수번호	교과목	담당교수	신청행위	신청
이수구분	학점	신청사건	학수번호	교과목	담당교수	신청행위	신청	
기초교양	2	1.1	CS10009	공조기동수	김학술	*****	신청	
전공필수	2	5.1	CS10013	알고리즘	문태준	*****	신청	

강좌신청 내역 (APPLIED COURSES)

이수구분	학점	신청사건	학수번호	교과목	담당교수	신청행위	삭제
전공선택	1	2.3	CS30004	자료구조심화	이도훈	*****	삭제
전공필수	3	2.5	CS30005	컴퓨터구조	정학술	*****	삭제
교양선택	1	4.3	CS30011	경제학의이해	박지훈	*****	삭제
전공필수	3	1.2	CS30001	개체지향프로그래밍	김지훈	*****	삭제
전공필수	2	1.4	CS30002	운영체제	박지훈	*****	삭제
기초교양	3	4.2	CS30010	심리학입문	황지민	*****	삭제
전공선택	2	3.1	CS30006	월호로그레팅	조세훈	*****	삭제

```
request.getRequestDispatcher("index.jsp").forward(request, response);

public int getTotalCredits() {
    open();
    int total = 0;
    String sql = "SELECT COALESCE(SUM(A_CREDITS), 0) FROM APPLIED_COURSES";

    try {
        pstmt = conn.prepareStatement(sql);
        ResultSet rs = pstmt.executeQuery();
        if (rs.next()) {
            total = rs.getInt(1);
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }

    return total;
}
```

제작과정 - 수강신청 페이지

- 기능

먼저 강남대학교의 수강신청 사이트를 참고해서 생성형 AI의 도움을 받아 CSS를 제작
회의를 통해 수강신청시 강의 평점을 한 눈에 볼 수 있도록 제작

학수번호를 입력해서 교과목을 조회하거나 학점을 필터링해 과목 검색 용이하게 제작

두개의 리스트(전체 강좌 목록, 강좌 신청내역)을 만들어서 신청을 누르면 전체 강좌
목록에서 삭제하고 강좌 신청내역으로 삽입, 삭제의 경우는 반대로 동작

제작과정 - 수강신청 페이지 - 실행화면

KANGNAM UNIVERSITY

2025년 웹프로그래밍 수강신청

이름 김유빈
학번 20220491
학과 소프트웨어
학년 2
주야 주간
최대신청학점 0 / 19

로그아웃

개설강좌신청

학수번호 예: CS...

확정

전체

조회

개설강좌신청 목록

이수구분	학점	강의시간	학수번호	교과목	담당교수	강의평점	신청
전공선택	1	2.3	CS30004	자료구조심화	이도윤	★★★★★	신청
전공필수	2	1.4	CS30002	운영체제	박서준	★★★★☆	신청
전공선택	2	4.1	CS30019	게임프로그래밍입문	박현성	★★★★☆	신청
전공필수	1	1.5	CS30017	소프트웨어공학	한유진	★★★★☆	신청
전공선택	3	3.3	CS30016	머신러닝	유성민	★★★★☆	신청
전공선택	3	2.1	CS30003	네트워크개론	최민혁	★★★★☆	신청
전공선택	3	5.4	CS30018	클라우드컴퓨팅	김나을	★★★★☆	신청
전공선택	2	2.4	CS30015	인공지능개론	조하림	★★★★☆	신청
전공선택	1	3.5	CS30008	정보보안개론	강지후	★★★☆☆	신청
교양선택	1	3.2	CS30020	생활영어회화	최라운	★★★★☆	신청

강좌신청 내역

신청학점(0)

시간표

이수구분	학점	강의시간	학수번호	교과목	담당교수	강의평점	삭제
개설신청 내역이 없습니다.							
division	credits	time	id	NAME	professor	rating	버튼

DB테이블

과목정보 테이블 생성

```
CREATE TABLE APPLIED_COURSES (  
  NUMBER      CHARACTER VARYING(10) NOT NULL,  
  A_DIVISION  CHARACTER VARYING(10) NOT NULL,  
  A_CREDITS   INT,  
  A_TIME      CHARACTER VARYING(10) NOT NULL,  
  A_ID        CHARACTER VARYING(10) NOT NULL,  
  A_NAME      CHARACTER VARYING(50) NOT NULL,  
  A_PROFESSOR CHARACTER VARYING(20),  
  A_RATING    CHARACTER VARYING(10)  
);
```

```
CREATE TABLE COURSES (  
  ID VARCHAR(10) PRIMARY KEY,  
  DIVISION VARCHAR(20),  
  CREDITS INT,  
  TIME CHARACTER VARYING(10) NOT NULL,  
  NAME VARCHAR(50),  
  PROFESSOR VARCHAR(20),  
  RATING VARCHAR(10)  
);
```

교과목 테이블에 데이터 추가

```
INSERT INTO COURSES(ID, DIVISION, CREDITS, TIME, NAME, PROFESSOR, RATING) VALUES  
A ('CS30001', '전공필수', 3, 1.2, '객체지향프로그래밍', '김지환', '★★★★★'),  
  ('CS30002', '전공필수', 2, 1.4, '운영체제', '박서준', '★★★★☆'),  
  ('CS30003', '전공선택', 3, 2.1, '네트워크개론', '최민혁', '★★★☆☆'),  
  ('CS30004', '전공선택', 1, 2.3, '자료구조심화', '이도윤', '★★★★★'),  
  ('CS30005', '전공필수', 3, 2.5, '컴퓨터구조', '정하늘', '★★★★☆'),  
  ('CS30006', '전공선택', 2, 3.1, '웹프로그래밍', '오세훈', '★★★★☆'),  
  ('CS30007', '전공선택', 3, 3.4, '모바일앱개발', '홍재민', '★★★★☆'),  
  ('CS30008', '전공선택', 1, 3.5, '정보보안개론', '강지훈', '★★★☆☆'),  
  ('CS30009', '기초교양', 2, 1.1, '글쓰기실습', '김하늘', '★★★★☆'),  
  ('CS30010', '기초교양', 3, 4.2, '심리학입문', '정다인', '★★★★★'),  
  ('CS30011', '교양선택', 1, 4.3, '경제학의이해', '박지훈', '★★★☆☆'),  
  ('CS30012', '교양선택', 3, 4.5, '철학의이해', '서은호', '★★★★☆'),  
  ('CS30013', '전공필수', 2, 5.1, '알고리즘', '문태준', '★★★★★'),  
  ('CS30014', '전공선택', 3, 5.2, '데이터마이닝', '이서율', '★★★★☆'),  
  ('CS30015', '전공선택', 2, 2.4, '인공지능개론', '조하림', '★★★★☆'),  
  ('CS30016', '전공선택', 3, 3.3, '머신러닝', '유성민', '★★★★☆'),  
  ('CS30017', '전공필수', 1, 1.5, '소프트웨어공학', '한유진', '★★★☆☆'),  
  ('CS30018', '전공선택', 3, 5.4, '클라우드컴퓨팅', '김나을', '★★★★☆'),  
  ('CS30019', '전공선택', 2, 4.1, '게임프로그래밍입문', '박현성', '★★★☆☆'),  
  ('CS30020', '교양선택', 1, 3.2, '생활영어회화', '최라온', '★★★☆☆');
```

제작과정 - 수강신청 - 필터링 기능

```
public List<Course> selectWhereAll(String id, String credits) {  
    open();  
    List<Course> cs = new ArrayList<>();  
    String sql;  
    ResultSet rs;  
  
    try {  
        if ("".equals(id)) {  
            sql = "SELECT * FROM COURSES WHERE CREDITS = ?";  
            pstmt = conn.prepareStatement(sql);  
            pstmt.setString(1, credits);  
        } else if ("NO".equals(credits)) {  
            sql = "SELECT * FROM COURSES WHERE ID = ?";  
            pstmt = conn.prepareStatement(sql);  
            pstmt.setString(1, id);  
        } else {  
            sql = "SELECT * FROM COURSES WHERE CREDITS = ? AND ID = ?";  
            pstmt = conn.prepareStatement(sql);  
            pstmt.setString(1, credits);  
            pstmt.setString(2, id);  
        }  
        rs = pstmt.executeQuery();  
    }  
}
```

교과목을 학점과 학수번호를
입력받아서 필터링하는 기능을
구현

제작과정 - 수강신청 페이지 -전체 데이터 조회

```
public List<Appl> yesApplication() {
    open();
    List<Appl> ap = new ArrayList<>();
    try {
        pstmt = conn.prepareStatement("SELECT * FROM APPLIED_COURSES");
        ResultSet rs = pstmt.executeQuery();

        while (rs.next()) {
            Appl a = new Appl();
            a.setNumber(rs.getString("NUMBER"));
            a.setId(rs.getString("A_ID"));
            a.setDivision(rs.getString("A_DIVISION"));
            a.setCredits(rs.getInt("A_CREDITS"));
            a.setTime(rs.getString("A_TIME"));
            a.setName(rs.getString("A_NAME"));
            a.setProfessor(rs.getString("A_PROFESSOR"));
            a.setRating(rs.getString("A_RATING"));
            ap.add(a);
        }
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
    return ap;
}
```

목적: 데이터베이스의 **APPLIED_COURSES** 테이블에 저장된 모든 강좌 기록을 조회합니다.

open(): 데이터베이스 연결을 설정합니다.

List<Appl> ap = new ArrayList<>(); 조회된 결과를 담은 Appl 객체 리스트를 초기화합니다. Appl 객체는 신청된 강좌의 상세 정보(학번, 학수번호, 학점 등)를 담는 데 사용됩니다.

객체 생성: 루프가 돌 때마다 새로운 Appl 객체(a)를 생성합니다.

데이터 매핑: ResultSet에서 컬럼 이름(예: "NUMBER", "A_ID")을 사용하여 데이터를 가져오고, 해당 값을 a 객체의 Setter 메서드(setNumber, setId 등)를 통해 설정합니다.

리스트 추가: 설정이 완료된 Appl 객체를 최종 결과 리스트 ap에 추가합니다.

제작과정 - 수강신청 페이지 - 신청버튼 입력시

두자

```
public void applyClass(App1 ap) {  
    if (ap.getId() != null) {  
        open();  
        String sql = "INSERT INTO APPLIED_COURSES VALUES(?,?,?,?,?,?,?)";  
        try {  
            pstmt = conn.prepareStatement(sql);  
            pstmt.setString(1, ap.getNumber());  
            pstmt.setString(2, ap.getDivision());  
            pstmt.setInt(3, ap.getCredits());  
            pstmt.setString(4, ap.getTime());  
            pstmt.setString(5, ap.getId());  
            pstmt.setString(6, ap.getName());  
            pstmt.setString(7, ap.getProfessor());  
            pstmt.setString(8, ap.getRating());  
  
            pstmt.executeUpdate();  
        } catch (Exception e) {  
            e.printStackTrace();  
        } finally {  
            close();  
        }  
    }  
}
```

매개변수: `App1 ap` 객체를 입력받으며, 이 객체에는 신청하려는 강좌의 모든 상세 정보가 담겨 있습니다.

유효성 검사: `if (ap.getId() != null)` 조건문을 사용하여, 신청 강좌의 학수번호(`id`)가 유효한지 먼저 확인합니다. 학수번호가 `null`이면 DB 작업이 수행되지 않습니다.

DB 연결: 유효성 검사를 통과하면 `open()` 메서드를 호출하여 데이터베이스 연결을 설정합니다.

SQL 쿼리: `APPLIED_COURSES` 테이블에 총 8개의 컬럼(`VALUES(?,?,?,?,?,?,?,?)`)에 값을 삽입하는 SQL 쿼리를 준비합니다.

제작과정 - 시간표 페이지

	월	화	수	목	금
1교시				게임프로그래밍입문 (박현성)	알고리즘 (문태준)
2교시					
3교시			머신러닝 (유성민)		
4교시					클라우드컴퓨팅 (김나율)
5교시					
전공선택	3	3.3	CS30016	머신러닝	유성민

```
<form action="/project/enrollment?action=applClass" method="post" style="margin: 0;">
  <input type="hidden" name="number" value="${sessionScope.userId}">
  <input type="hidden" name="division" value="${course.division}">
  <input type="hidden" name="credits" value="${course.credits}">
  <input type="hidden" name="time" value="${course.time}">
  <input type="hidden" name="id" value="${course.id}">
  <input type="hidden" name="name" value="${course.name}">
  <input type="hidden" name="professor" value="${course.professor}">
  <input type="hidden" name="rating" value="${course.rating}">
  <button type="submit" class="apply-btn">신청</button>
</form>
```

index.jsp부분

```
case "applClass":
    // 신청버튼 눌렀을때
    request.setAttribute("loglog", daolog.getUserById(loginUserID)); //로그인정보
    applClass(request, response); // 강좌신청 목록에서 -> 강좌신청 내역으로 이동(INSERT)
    deleteAllList(request, response); // 강좌신청목록(COURSE)에서 삭제
    allList(request, response);
    yesList(request, response);
    request.setAttribute("totalCount", dao.getTotalCredits()); // 신청확정
    request.getRequestDispatcher("index.jsp").forward(request, response);
    break;

case "deleteClass":
    // 삭제버튼 눌렀을때
    request.setAttribute("loglog", daolog.getUserById(loginUserID));
    allApplClass(request, response); // 강좌신청내역에서 -> 목록으로(INSERT)
    deleteYesApplication(request, response); // 강좌신청내역(APPLIED_COURSES)에서 삭제
    allList(request, response);
    yesList(request, response);
    request.setAttribute("totalCount", dao.getTotalCredits());
    request.getRequestDispatcher("index.jsp").forward(request, response);
    break;
```

Servlet부분

전공선택	3	5.2	CS30014	데이터마이닝	이서율	★★★★☆	신청
전공선택	2	2.4	CS30015	인공지능개론	조하림	★★★★☆	신청
전공필수	1	1.5	CS30017	소프트웨어공학	한유진	★★★★☆	신청
교양선택	1	3.2	CS30020	생활영어회화	최라온	★★★★☆	신청

강좌신청 내역

신청확정(10)

시간표

이수구분	학점	강의시간	학수번호	교과목	담당교수	강의평점	삭제
전공선택	3	3.3	CS30016	머신러닝	유성민	★★★★☆	삭제
전공선택	3	5.4	CS30018	클라우드컴퓨팅	김나율	★★★★☆	삭제
전공선택	2	4.1	CS30019	게임프로그래밍입문	박현성	★★★★☆	삭제
전공필수	2	5.1	CS30013	알고리즘	문태준	★★★★★	삭제

제작과정 - 시간표 페이지

DAO부분

전공선택	3	5.2	CS30014	데이터마이닝	이서율	★★★★☆	신청
전공선택	2	2.4	CS30015	인공지능개론	조하림	★★★★☆	신청
전공필수	1	1.5	CS30017	소프트웨어공학	한유진	★★★★☆	신청
교양선택	1	3.2	CS30020	생활영어회화	최라운	★★★★☆	신청

강좌신청 내역

신청확정(10)

시간표

이수구분	학점	강의시간	학수번호	교과목	담당교수	강의평점	삭제
전공선택	3	3.3	CS30016	머신러닝	유성민	★★★★☆	삭제
전공선택	3	5.4	CS30018	클라우드컴퓨팅	김나을	★★★★☆	삭제
전공선택	2	4.1	CS30019	게임프로그래밍입문	박현성	★★★★☆	삭제
전공필수	2	5.1	CS30013	알고리즘	문태준	★★★★★	삭제

```
// 신청버튼 눌렀을때 강좌신청내역으로 insert
public void applClass(Appl ap) {
    if (ap.getId() != null) {
        open();
        String sql = "INSERT INTO APPLIED_COURSES VALUES(?,?,?,?,?,?,?,?)";
        try {
            pstmt = conn.prepareStatement(sql);
            pstmt.setString(1, ap.getNumber());
            pstmt.setString(2, ap.getDivision());
            pstmt.setInt(3, ap.getCredits());
            pstmt.setString(4, ap.getTime());
            pstmt.setString(5, ap.getId());
            pstmt.setString(6, ap.getName());
            pstmt.setString(7, ap.getProfessor());
            pstmt.setString(8, ap.getRating());

            pstmt.executeUpdate();
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            close();
        }
    }
}

// 신청버튼 누르면 전체강좌에서 목록 delete
public void deleteAllList(Course c) {
    open();
    String sql = "DELETE FROM COURSES WHERE ID = ?";
    try {
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, c.getId());
        pstmt.executeUpdate();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        close();
    }
}
```

제작과정 - 시간표 페이지

	월	화	수	목	금
1교시				게임프로그래밍입문 (박현성)	알고리즘 (문태준)
2교시					
3교시			머신러닝 (유성민)		
4교시					클라우드컴퓨팅 (김나을)
5교시					
전공선택	3	3.3	CS30016	머신러닝	유성민

```

<script>
// 시간표 모달 열기
function openTimetable() {
    document.getElementById("timetableModal").style.display = "block";
}
// 시간표 모달 닫기
function closeTimetable() {
    document.getElementById("timetableModal").style.display = "none";
}
//바깥영역 클릭시 모달 닫기
window.onclick = function(e){
    let modal = document.getElementById("timetableModal");
    //클릭한 대상이 모달의 바깥이라면 -> 모달 닫기
    if(e.target === modal){
        modal.style.display = "none";
    }
}
</script>

```

```

<tbody>
<%
// 1) 5일 x 5교시 형태의 시간표 데이터 저장용 배열 생성
// (실제 배열은 1부터 사용하기 위해 6x6 으로 선언)
String[][] table = new String[6][6];
// 2) 현재 신청된 강좌 리스트 가져오기 (Controller에서 전달)
java.util.List<project.App1> list =
    (java.util.List<project.App1>)request.getAttribute("appliedCourses");
// 3) 리스트를 반복하면서 각 강의의 시간(time)을 분석
// ex) "2.3" -> 2요일, 3교시
if(list != null){
    for(project.App1 c : list){
        try{
            // ex) "2.3"
            String time = c.getTime();
            // 문자열을 '.' 기준으로 분리 '2.3'을 문자 그대로의미하기 위해 앞에 \\를 사용
            String[] parts = time.split("\\.");
            // 요일, 교시 숫자로 변환(월부터1~5, 1교시~5교시)
            int day = Integer.parseInt(parts[0]);
            int period = Integer.parseInt(parts[1]);
            // 배열 범위 내인지 검사 후 저장
            if(day>=1 && day<=5 && period>=1 && period<=5){
                table[period][day] = c.getName() + "<br>" + c.getProfessor() + " ";
            }
        }catch(Exception e){}
    }
}
// 4) 완성된 2차원 배열을 시간표 형태로 HTML로 출력
for(int i=1;i<=5;i++){
    %>
    <tr>
    <!-- 교시 index 출력 -->
    <th><%=i>교시</th>

    <% for(int j=1;j<=5;j++){ %>
    <!-- 해당 칸에 데이터 있으면 표시, 없으면 빈칸 -->
    <td><%= table[i][j] == null ? "" : table[i][j] %></td>
    <% } %>
    </tr>

    <% } %>
</tbody>

```

index.jsp

게임프로그래밍입문
(박현성)

어려웠던 점

1.DB를 사용시 파일의 위치와 사용자명이 상이해서 반복적인 오류 발생

-데이터 베이스 정보를 모두 같은 조건이 되도록 새로 만들어서 수정

2.각자 만들 페이지를 회의로 통해 결정했는데 결정 과정에서 변수명이나 설정에 대한 이야기를 나누지 않아서 서로 다른 변수명과 파일이 많아져 통합을 하는것에 어려움이 있었다.

-각 부분에서 필요한 부분만 가져와서 수정 후 새 프로젝트를 만들어서 공유

향후계획

실시간으로 서버의 시간을 파악해서 사이트에서 알람이 울리도록 하는 기능을 추가 예정.(네이비즘 시계)

코드를 최적하여 3명이서 만든 프로젝트의 중복된 코드나 함수를 제거하고 가독성을 높일 예정이다.

이외에도 자신 말고 다른 사람의 시간표도 확인하고 공유할 수 있는 기능도 추가하고

강의별로 직접 평점을 입력할수 있는 기능도 추가하고자 한다.

시연영상